

A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a stylized tree structure.

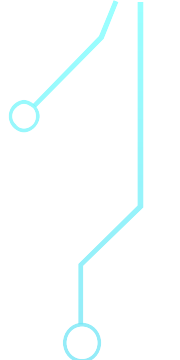
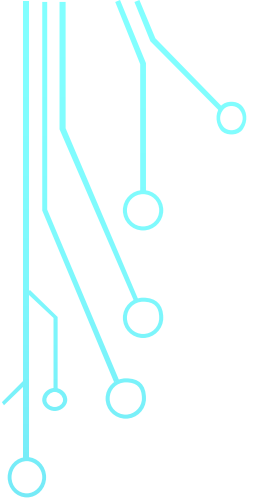
Unix Basics

COMP211 Connor McMahon

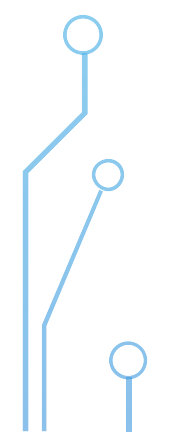
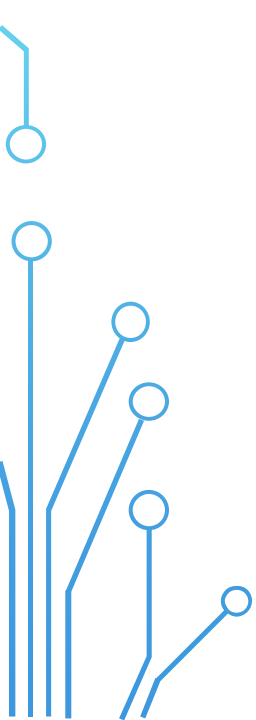
Quiz 1 is Friday

- See Canvas for
 - Logistics
 - Learning Objectives
 - Practice Problems
- Tutoring on Monday and Tuesday 5-6pm in the CSXL
- TA-led review session on Wednesday 6-7:30pm in FB141

Finish Debugging Exercise from Last Lecture



Shell Prompt Configuration



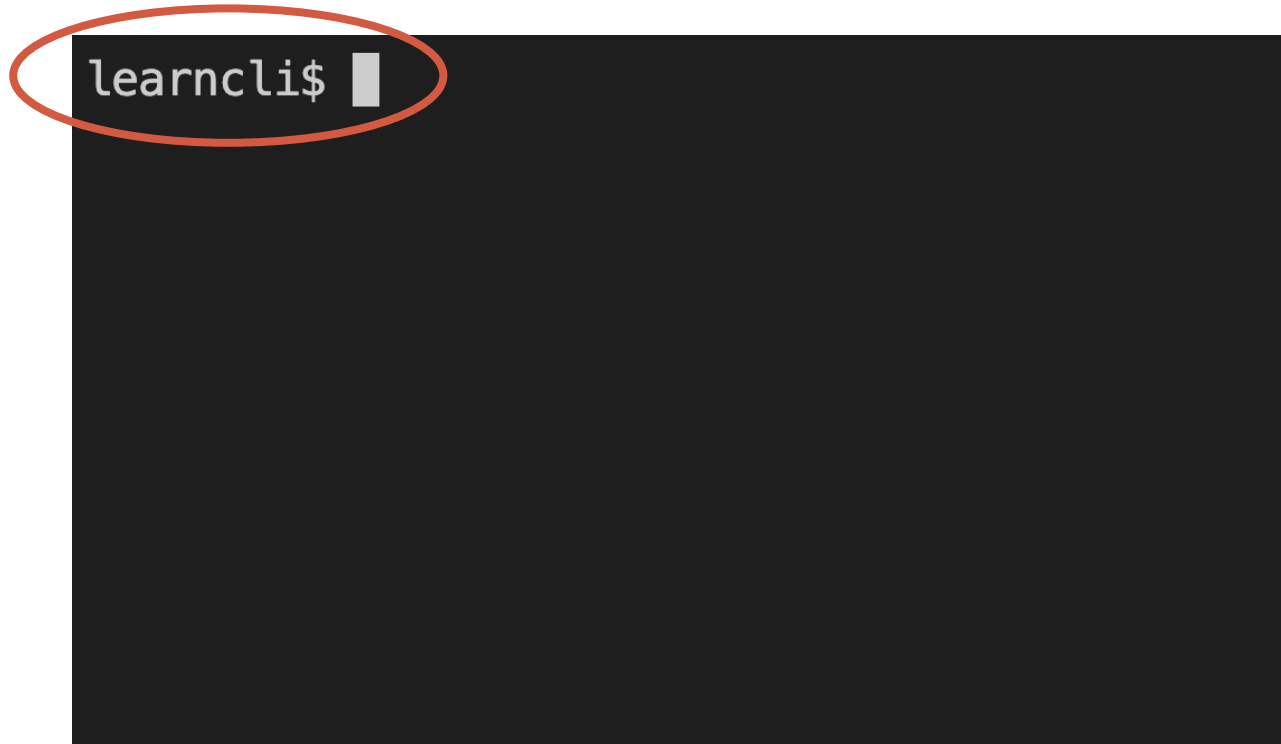
Command-line Interface: A text-based way to interact with a system

Terminal: The app/window you type into

Shell: The program that understands and runs your commands

Recall: Shell Prompt

- **Shell prompt:** It's the shell's way of saying, "I'm ready for you to give me a command."



What does \$ mean in instructions?

- When you see instructions like this:
 - `$ echo "hello, world"`
- The \$ is not part of the command.
- It represents the command prompt.
- The \$ symbol tells you:
 - “The computer is ready for you to type a command.”
- So:
 - You do not type the \$
 - You only type what comes after it

Why do I sometimes see `learncli$` instead of `$`?

- Different systems show different prompts.
- `learncli$`:
 - `learncli` is the name of the container
- `cece@mbp $`
 - Username: cece
 - Machine name: mbp
- You can control what your shell prompt displays
 - The shell prompt is controlled by a variable named `PS1`.
 - You can see it with `echo $PS1`, and change it by setting `PS1` to a new value.

Shell Prompt Configuration Example

```
cece@Cecce-MacBook-Pro grades % echo $PS1
%n@%m %1~ %#
cece@Cecce-MacBook-Pro grades %
```

Part	Meaning
%n	username (cece)
@	Literal @ character
%m	machine (host) name (ceces-mbp)
	A space
%1~	The current directory, shortened to the last component (grades)
	A space
%#	The prompt symbol (% for normal user, # if root)

Environment Variables

Environment Variables

- An **environment variable** is a named value that a program can receive from the environment in which it runs.
- Environment variables are commonly used to give programs information about their configuration or surroundings.
- Example:
 - `export COURSE="COMP 211"`
 - If you start a program from this shell, that program can access the value of **COURSE**.

Common Environment Variables

Variable	Description
PATH	Directories the shell searches to find commands.
HOME	Path to the user's home directory.
USER	Current user's username.
SHELL	User's configured login shell.
OLDPWD	Previous working directory; used by cd -.
PS1	Controls the appearance/content of the primary interactive shell prompt.

See Your Environment Variables

- You can see the environment variables available to your shell with
 - `printenv` or `env`

Example

```
$ printenv  
HOME=/home/student  
USER=student SHELL=/bin/bash  
TERM=xterm-256color  
PATH=/usr/local/bin:/usr/bin:/bin
```

Read an Environment Variable

- In Bash, \$ means “give me the value stored in this variable.”

```
$ echo $HOME  
/home/student
```

```
$ echo ${HOME}  
/home/student
```

- `$VAR` and `${VAR}` both access the value of a variable. The braces make it explicit where the variable's name begins and ends.
- Without the \$, the above command would just print the literal text

```
$ echo HOME  
home
```

Change a Variable

- You can assign a new value with =:

```
learncli$ EDITOR=vim
```

- Don't put spaces around =.

```
learncli$ EDITOR=vim      # correct
```

```
learncli$ EDITOR = vim    # incorrect
```

- But at this point, EDITOR is only a **shell variable**.
- To make it an **environment variable** that programs launched from this shell can inherit:

```
learncli$ export EDITOR
```

- Or do both at once:

```
learncli$ export EDITOR=vim
```

Modify an Existing Variable

- Sometimes you want to add to an existing value rather than replace it.
- Suppose:

```
$ echo "$PATH"
```

- shows:

```
/usr/local/bin:/usr/bin:/bin
```

- and you want the shell to also search \$HOME/bin. You could write:

```
$ export PATH="$PATH:$HOME/bin"
```

- Bash first expands \$PATH and \$HOME, producing something like:

```
/usr/local/bin:/usr/bin:/bin:/home/student/bin
```

Remove a Variable

- Use unset:

```
$ unset EDITOR
```

- Now:

```
$ echo "$EDITOR"
```

- prints nothing.

What is a Child Shell?

- When you type:

```
$ bash
```

- you start another Bash shell from inside your current Bash shell.
- We call the new shell a **child shell**, and the shell that started it is its **parent**.
- A child process inherits **environment variables** from its parent.
- However, ordinary **shell variables** **are not** passed to the child.

Child Shell

- When you're finished with the child shell, run:

```
$ exit
```

- The child shell terminates, and you're back in the original parent shell.
- **Remember, a shell is just a program.** When you type `bash`, your current Bash process launches another instance of the Bash program.
- That new Bash instance is therefore a child **process** of the original Bash process.

Active Learning

- First, explore some variables that are already part of your environment:
 - `$ echo "$SHELL"`
 - `$ echo "$TERM"`
 - `$ echo "$PATH"`
- Now create a new **shell variable**:
 - `$ COURSE="COMP 211"`
 - `$ echo "$COURSE"`
- Next, create an **environment variable**:
 - `$ export EDITOR="vim"`
 - `$ echo "$EDITOR"`
- Continue to next slide

Active Learning

- Start a child Bash shell:
 - `$ bash`
- Now try:
 - `$ echo "$COURSE"`
 - `$ echo "$EDITOR"`
- Then return to your original shell:
 - `$ exit`
- Why could the child shell access one variable but not the other?

Changes Are Not Permanent

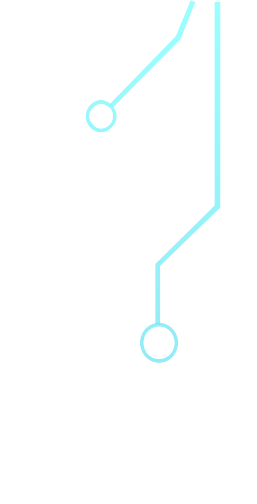
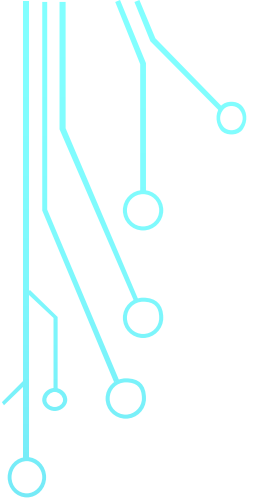
- If you type:

```
$ export EDITOR=nano
```

- you've changed the environment of your **current shell**. If you close that shell and start a new one, your change may disappear.
- If you want Bash to make the change whenever you start an interactive shell, you can put the command in `~/ .bashrc`:

```
export EDITOR=nano
```

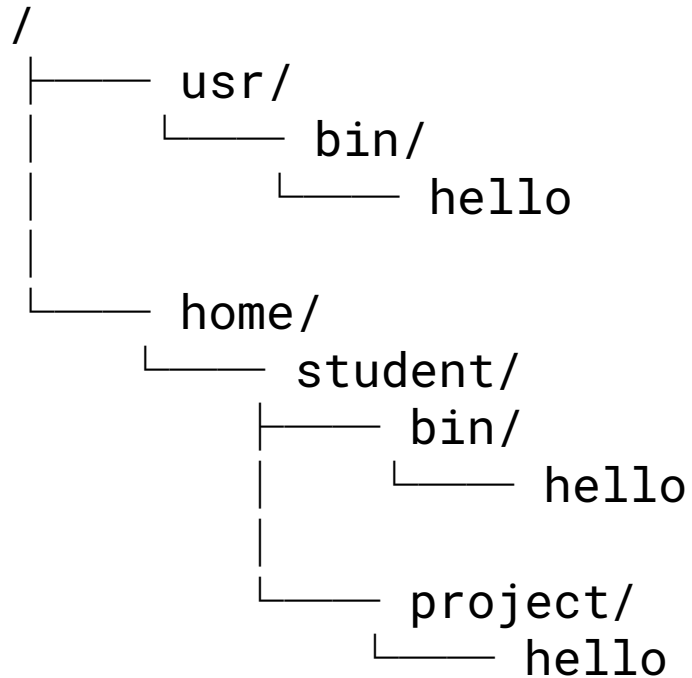
- Then a new Bash session will execute that command when it reads `.bashrc`.



\$PATH

Example

- You are currently in: `/home/student/project`
- Your `$PATH` is: `/home/student/bin:/usr/bin:/bin`
- Your filesystem contains:



Command	Which hello runs?
<code>./hello</code>	<code>/home/student/project/hello</code>
<code>/usr/bin/hello</code>	<code>/usr/bin/hello</code>
<code>../bin/hello</code>	<code>/home/student/bin/hello</code>
<code>hello</code>	<code>/home/student/bin/hello</code>

- All three hello files are executable.

What is the Default Value of \$PATH?

- `$PATH` usually contains several standard directories
- For example, a Linux system might have:

`/usr/bin:/bin`

- What do these directories contain?

Directory	Purpose	Examples
<code>/usr/bin</code>	Many standard system commands	<code>gcc</code> , <code>python3</code> , <code>git</code>
<code>/bin</code>	Essential system commands	<code>ls</code> , <code>cp</code> , <code>mv</code> , <code>cat</code> , <code>mkdir</code> , <code>pwd</code> , <code>rm</code>

The which command

- This tells you which executable will be found through \$PATH.

```
learncli$ which wc  
/usr/bin/wc  
learncli$ which python3  
/usr/bin/python3  
learncli$ which which  
/usr/bin/which
```

How to Modify Path

- **Add a directory to the beginning**

```
export PATH="/home/student/bin:$PATH"
```

- Now the shell searches:

`home/student/bin`, then existing PATH directories

- Putting a directory first gives programs there **higher priority**.

- **Add a directory to the end**

```
export PATH="$PATH:/home/student/bin"
```

- Now the shell searches the existing directories first and `/home/student/bin` last.

Example

- Suppose:

```
$ echo $PATH
```

```
/usr/local/bin:/usr/bin:/bin
```

- Then:

```
$ export PATH="/home/student/bin:$PATH"
```

```
$ echo $PATH
```

```
/home/student/bin:/usr/local/bin:/usr/bin:/bin
```

Be careful about removing existing contents!

- This:

```
export PATH="/home/student/bin:$PATH"
```

- **adds** /home/student/bin while preserving the old \$PATH.

- But this:

```
export PATH="/home/student/bin"
```

- **replaces the entire \$PATH**, which can cause commands like `ls`, `cat`, and `grep` to stop being found.

\$PATH

- Suppose your current PATH is:
 - `/usr/local/bin:/usr/bin:/bin`
- and your home directory is:
 - `/home/student`
- You run:
 - `export`
`PATH="${PATH}:/home/student/mybin"`
- What is the new value of PATH?

- Example Pipeline

- `./producer | sort | ./consumer | wc -l`

- For each program, determine how Bash finds the executable

Command	Search \$PATH or use specified path?
<code>./producer</code>	
<code>sort</code>	
<code>./consumer</code>	
<code>wc</code>	

\$PATH



- Suppose your current PATH is:

```
/usr/local/bin:/usr/bin:/bin
```

- and your home directory is:

```
/home/student
```

- You run:

```
export  
PATH="${PATH}:/home/student/mybin"
```

- What is the new value of PATH?

```
/usr/local/bin:/usr/bin:/bin:  
/home/student/mybin
```

- Example Pipeline

```
./producer | sort | ./consumer | wc -l
```

- For each program, determine how Bash finds the executable

Command	Search \$PATH or use specified path?
./producer	Specified path
sort	Search \$PATH
./consumer	Specified path
wc	Search \$PATH

Temporary \$PATH Changes

- If you run:

```
export PATH="$PATH:/home/student/bin"
```

- the change affects the **current shell session**.
- You can verify it with:

```
echo $PATH
```
- But when you **close the terminal and open a new one**, the change is generally gone.

Persistent \$PATH Changes

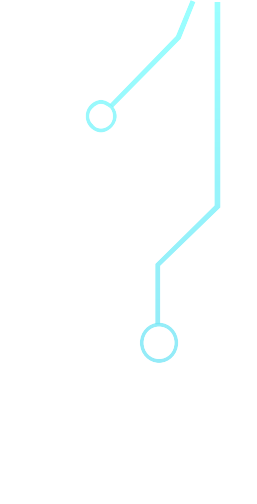
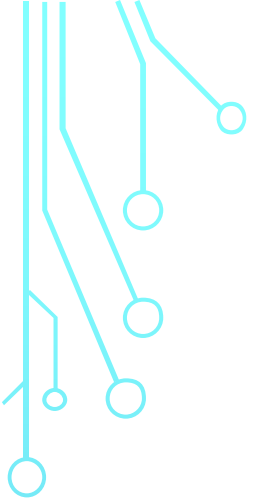
- To make the change apply to **future shell sessions**, put the command in a shell configuration/startup file.

- Example: adding a line to `.bashrc`:

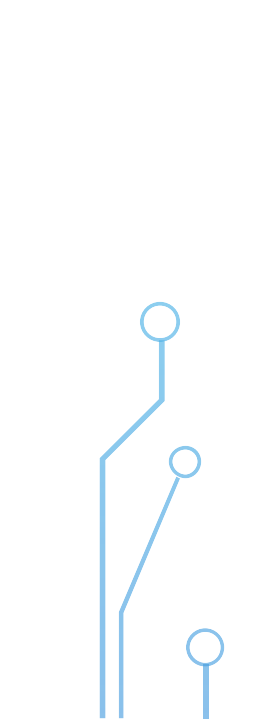
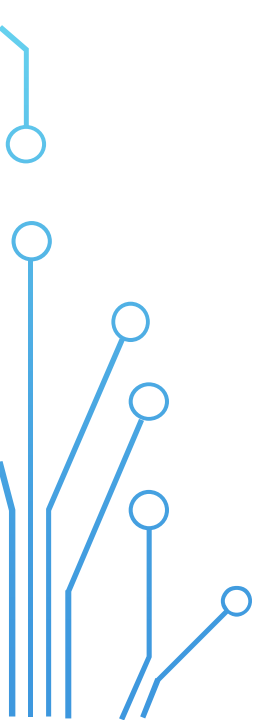
```
export PATH="$PATH:/home/student/bin"
```

- Then new applicable Bash sessions can load that setting. You can also reload that file in the current shell with:

```
source ~/.bashrc
```



`.bashrc`



.bashrc

- Your **.bashrc** is a configuration file for Bash. It contains commands that Bash runs when you start a new interactive Bash shell.
- The file is located in your **home directory**:
 - `~/ .bashrc`
- The `.` at the beginning of the filename makes it a **hidden file**, so it normally won't appear with a plain `ls`. You can see it with:

```
$ ls -a ~
```

.bashrc

- Your `.bashrc` can be used to customize how your shell looks and behaves. For example, it might contain:

```
# Change the prompt
PS1="learncli$ "

# Create a shortcut
alias ll="ls -al"

# Set an environment variable
export EDITOR="vim"

# Add a directory containing programs to PATH
export PATH="${PATH}:${HOME}/bin"
```

Alias

- An **alias** lets you create a shortcut for a shell command.
- For example, if you frequently type:

```
ls -al
```

- you could create an alias:

```
alias ll='ls -al'
```

- Another practical one is:

```
alias c='clear'
```

- Then:

```
c
```

- clears the terminal.

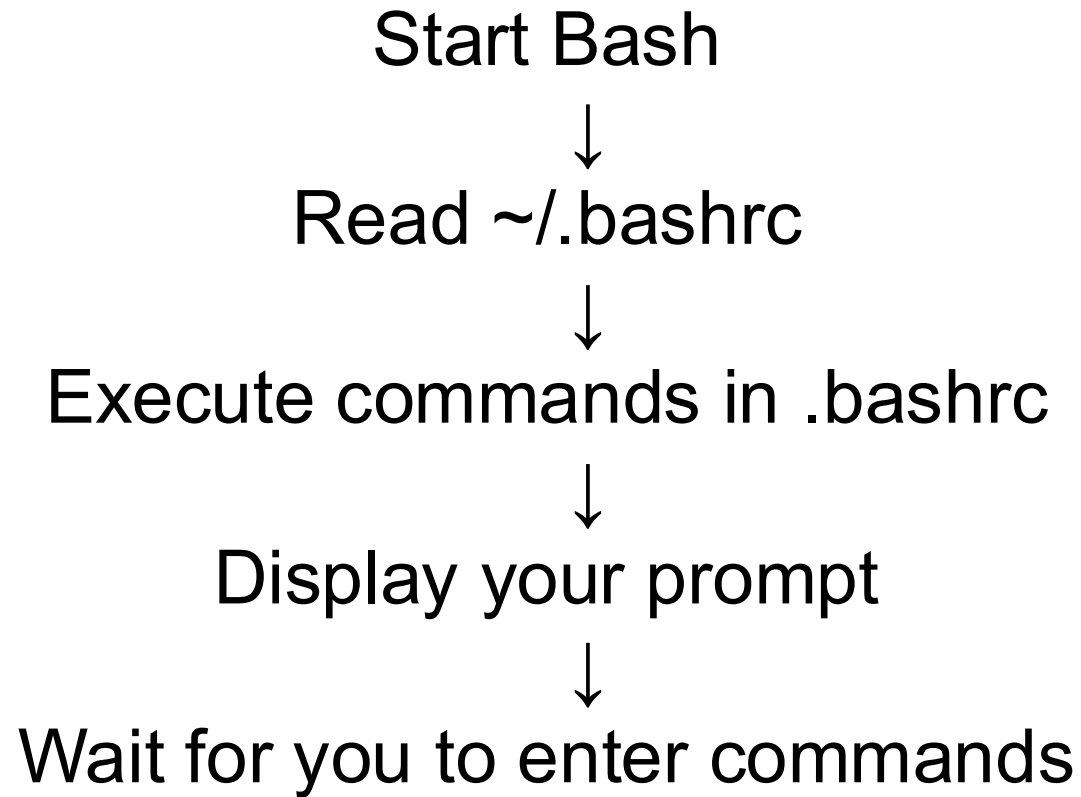
.bashrc

- An important thing to remember is that `.bashrc` is not just a list of settings.
- It contains Bash commands. When Bash reads `.bashrc`, it executes those commands.
- If you modify `.bashrc`, the changes generally won't affect a shell that's already running. You can either open a new Bash shell or tell the current shell to read the file again:

```
$ source ~/.bashrc
```
- The `source` command tells Bash to read a file and execute the commands in that file in the current shell.

.bashrc

- So when you open a terminal and Bash starts, the sequence is as follows:

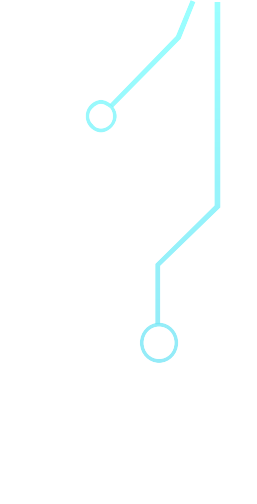
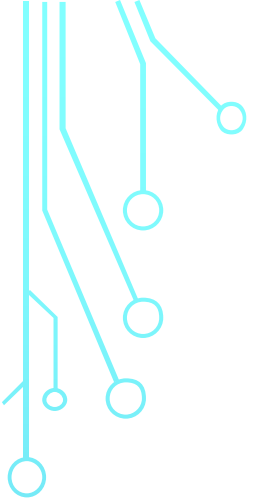


source

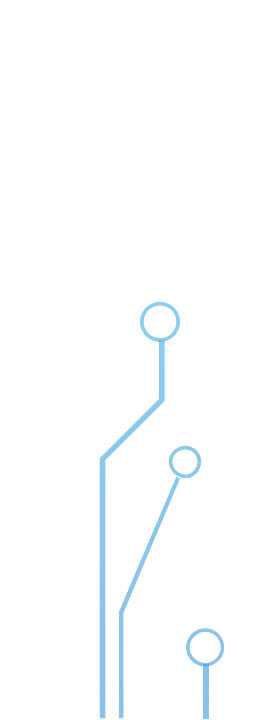
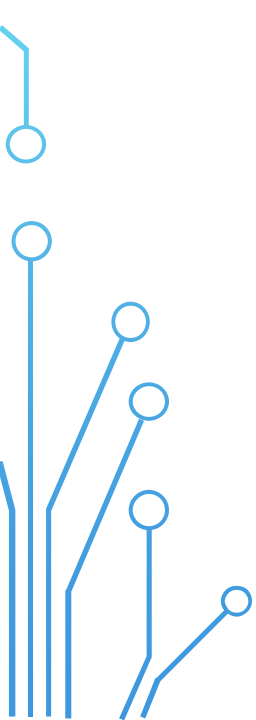
- You've probably seen source before
- When working with Python, you may have created a virtual environment:

```
python3 -m venv .venv
```
- and then activated it with:

```
source .venv/bin/activate
```
- The activate file is actually a **shell script** containing commands that modify your current shell.



Python Virtual Environments

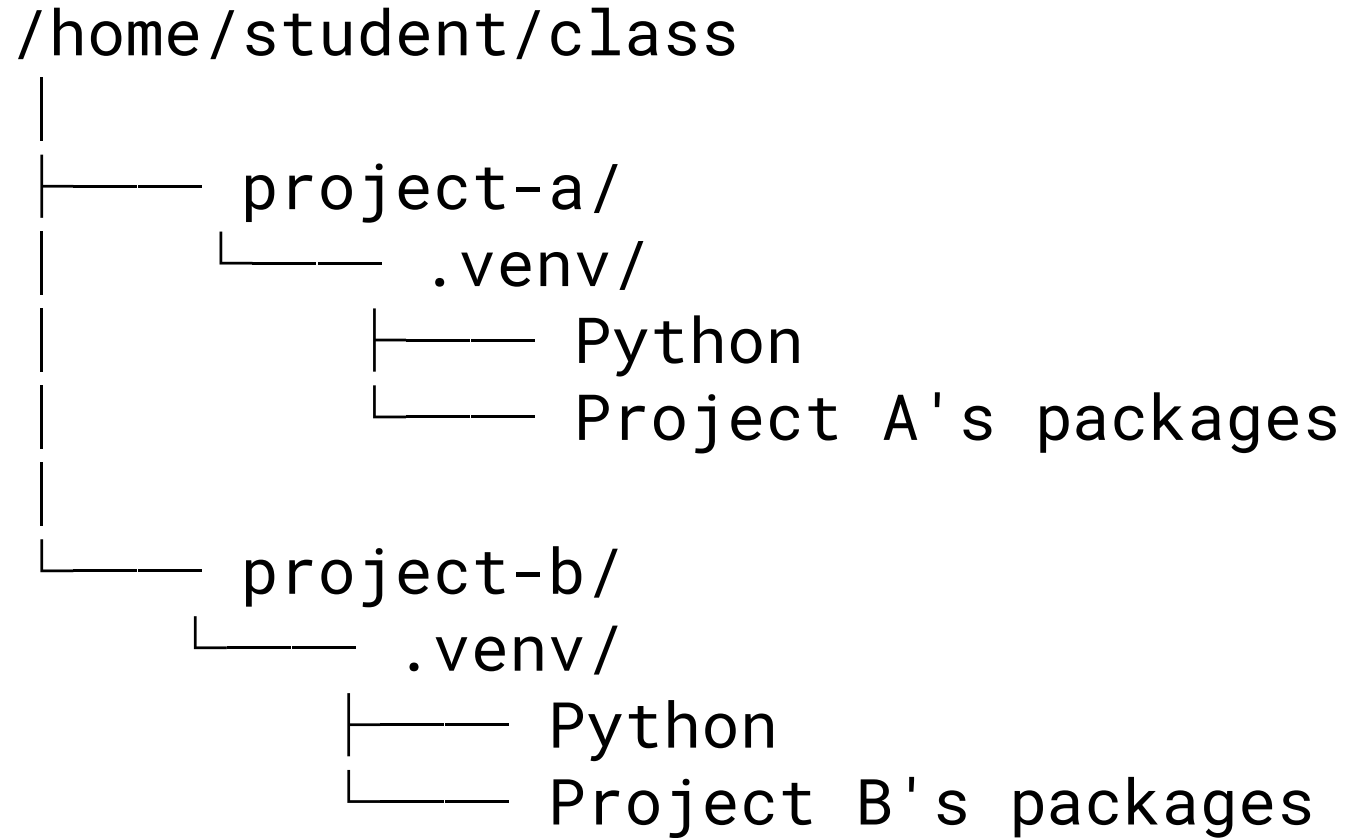


Python Virtual Environments

- A Python virtual environment is an isolated environment for a Python project that has its own Python executable and its own set of installed Python packages.
- Why do we use them? Imagine you have two projects:
 - Project A needs package version 1
 - Project B needs package version 2
- If every Python project uses the same globally installed packages, these requirements can conflict.

Python Virtual Environment

- Virtual environments give each project its own place for packages:



Creating a Virtual Environment

- The following creates a directory called `.venv` containing the virtual environment.

```
python3 -m venv .venv
```

- Then the following activates it:

```
source .venv/bin/activate
```

- After activation, commands such as:
 - `python`
 - `pip`
- refer to the versions inside `.venv`.

Python Virtual Environments

- You can investigate this with:

```
$ which python
```

- Before activation, you might see:

```
$ /usr/bin/python
```

- After activation:

```
$ /home/student/project/.venv/bin/python
```

What's actually happening?

- Activation modifies your shell's environment (most importantly, **\$PATH**) so that the virtual environment's `bin` directory comes **before** the directories containing the system Python.
- A Python virtual environment generally **doesn't contain a complete independent copy of Python**.
 - The `python` inside `.venv/bin` is commonly a **link to an existing Python interpreter**, rather than another full copy of Python.
 - The virtual environment mainly provides an isolated location for the project's installed packages.